Implementation Guide: the safe combo `zono` + `prev_pgd` + $\tau = 0.5$, with hyper-attack and VHAGaR-deps wiring

Code-level companion to `advstd_techniques.tex`

2026-04-25

Abstract

This document is a code-level walkthrough of three things: (1) the top-ranked safe combination `zono` + `prev_pgd` + $\tau = 0.5$ from Section 9 of `advstd_techniques.tex`; (2) the hyper-attack (PGD warm-start) integration into advstd Phase 2; (3) the VHAGaR dependency constraints (`--activate_vaghgar_deps`) as they apply to advstd Phase 2. The aim is operational: every claim is backed by a file path and line range, and the call order in `main_advanced_standard` (`run.jl`) is reproduced verbatim so a reader can step through it.

The combo is sound: at solver `OPTIMAL` it returns δ_{exact}. The other two pieces (hyper-attack and VHAGaR-deps) are independent toggles that work alongside the combo. Hyper-attack provides a Cutoff and (in `prev_pgd` mode) a warm-start that the consensus filter then rationalises against N_1. VHAGaR-deps adds inter-copy constraints linking the org and pert encodings and is preserved exactly as in standard mode.

Contents

1 Code Map

All paths relative to `vaghar_org/`.

Topic	File	Key entry points (line)
Phase-2 driver	`run.jl`	`main_advanced_standard` (630–1052)
Phase-2 N2-only wrapper	`run.jl`	`main_advanced_standard_n2` (1181–1188)
Tech 1 zono Source A	`utils/perturbation_intervals.jl`	`compute_diff_bounds_zonotope` (1317–1740)
Tech 1 zono Source B	`utils/perturbation_intervals.jl`	`compute_n2_zono_abs_bounds` (1743–1988)
Tech 1 bound integration	`utils/MIPVerify.jl/.../core_ops.jl`	`intersect_per_copy_bounds` (199–246)
Tech 2 prev_pgd hints	`utils/help_functions.jl`	`apply_n1_var_hints!` (598–741)
Tech 2 p-formula	`utils/help_functions.jl`	`compute_varhint` (507–523)
Tech 3 τ decision	`utils/n2_relax_decision.jl`	`compute_n2_relax_decision!` (23–130)
Tech 3 MIP emit	`utils/MIPVerify.jl/.../core_ops.jl`	`relu` (248–610), gating at 554–605
Hyper-attack subprocess	`utils/hyper_attack.jl`	`hyper_attack` (98–113)
Hyper-attack PGD code	`utils/hyper_attack.py`	`attack` (120–303)
Hyper-attack hint apply	`utils/hyper_attack.jl`	`hyper_attack_hints` (1–35)
Cutoff plumbing	`utils/mip.jl`	`mip_set_attr` (20–34)
VHAGaR deps (entry)	`utils/perturbation_dependencies.jl`	`perturbation_dependencies` (202–232)
VHAGaR deps (encoder)	`utils/perturbation_dependencies.jl`	`encode_dependencies` (112–200)

2 Phase-2 call sequence (the source of truth)

The advstd Phase-2 driver `main_advanced_standard` in `run.jl:630--1052` processes one (c_s, c_t) pair at a time. Globally (before the pair loop), it loads the `n1_state` directory and calls `compute_diff_bounds_zor` + `compute_n2_zono_abs_bounds` once.[1] Per pair, the order is:

1. Run `hyper_attack` for N_2 to get `suboptimal_solution_n2` for the Gurobi `Cutoff`. `run.jl:909--912`
2. `mip_reset()`; install N_1's neuron bounds globally via `set_n1_neuron_bounds(n1_layers_info)`. `run.jl:930`
3. *(Tech 3, only if $\tau \geq 0$)* call `compute_n2_relax_decision!(`τ`)` — this populates the `n2_relax_decision` dict consulted later inside `relu()`. `run.jl:945--949`
4. `get_model(...)` builds the dual-copy N_2 MIP. Inside every `relu()` call, both the per-copy bound integration (Sources A/B/C) and the triangle-vs-bigM choice happen here. `run.jl:951--956`

[1] Grep `run.jl` around lines 1090–1100 for the once-only zonotope computation. The Source-B pass is genuinely shared across (c_s, c_t) because it depends only on N_2's weights and the input box.

5. Build the result filename suffix from active flags. For our combo: `_BoundTightPertRelax0.5_zonoBounds_va` `run.jl:962--989`
6. *(hyper-attack)* `hyper_attack_hints(m_n2, token*"_n2", c_tag, c_target)` — read PGD output from `/tmp` and call `set_start_value` on the matched binaries. Filename suffix `_HyperAttackHints` appended. `run.jl:999--1006`
7. *(VHAGaR deps)* `perturbation_dependencies(m_n2, nn2, ...)`; suffix `_VagharDeps`. `run.jl:1007--101`
8. *(perturbed intervals)* `perturbed_interval_constraints(m_n2, nn2, "org", "perturbation")`; suffix `_PerturbedIntervals`. `run.jl:1012--1015`
9. `mip_set_delta_property`, `set_optimizer`, `mip_set_attr` (this is where the `Cutoff` is installed using the value from step 1). `run.jl:1016--1018`
10. *(Tech 2)* call `apply_n1_var_hints!(m_n2, var_hint_mode, ...)` — must run *after* `hyper_attack_hints` so the Start-consensus filter sees PGD's values. `run.jl:1027--1034`
11. `optimize!(m_n2)`, then `mip_log`. `run.jl:1043--1044`

The order matters. Two specific dependencies:

- `compute_n2_relax_decision!` must run *before* `get_model`, because `relu()` consults the global decision dict at MIP-build time. Calling it after the build is a no-op.
- `apply_n1_var_hints!` (the `prev_pgd` branch) must run *after* `hyper_attack_hints`, otherwise PGD silently overwrites the consensus filter's writes.

3 Tech 1 — `zono` bound tightening

3.1 Source A — diff zonotope $N_1 \to N_2$

What it produces. For every ReLU neuron (m, k) in N_2, two arrays: `relu_diff_up_bounds[m][k]` and `relu_diff_down_bounds[m][k]`, sound over-approximations of $d^{N_2}_{m,k} - d^{N_1}_{m,k}$. These are stored as 1-indexed Julia arrays of length K = number of ReLU layers per copy (`help_functions.jl`, declared near line 74).

Where. `compute_diff_bounds_zonotope` in `utils/perturbation_intervals.jl:1317--1740`. The affine update at lines 1397–1423 implements

$$c_{m,k} \leftarrow \sum_{k'} w^2_{m,k,k'}\, c_{m-1,k'} + \sum_{k'} \Delta w_{m,k,k'}\, \mathrm{mid}_{k'} + \Delta b_{m,k},$$

$$G'_{\mathrm{corr}} = W_2 \cdot G_{m-1}, \qquad G'_{\mathrm{indep}} = \Delta W \cdot \mathrm{diag}(\mathrm{rad}_{1..K'}),$$

$$G_m \leftarrow [G'_{\mathrm{corr}} \mid G'_{\mathrm{indep}}],$$

with $\mathrm{mid}_{k'} = \frac{1}{2}(\max(0, u^{N_1}_{m-1,k'}) + \max(0, l^{N_1}_{m-1,k'}))$ and $\mathrm{rad}_{k'}$ the half-width (post-ReLU clipping is applied, matching `advstd_techniques.tex` §4.1.2). The DeepZ ReLU update (formulas $\lambda_k = u_k/(u_k - l_k)$, $\mu_k = -u_k l_k/(2(u_k - l_k))$) lives at `perturbation_intervals.jl` lines 1614–1615; the per-split-neuron new column with μ_k at row k and zero elsewhere is appended at line 1958.

Caching. Source A is computed exactly once per Phase-2 invocation, before the (c_s, c_t) loop. The resulting global arrays are then reused for every pair.

3.2 Source B — absolute N_2 zonotope

What it produces. `n2_abs_up_bounds[m][k]` and `n2_abs_down_bounds[m][k]` — sound over-approximations of $\hat{z}^{N_2}_{m,k}$ propagated through N_2 alone, with a single zonotope whose initial hull covers *both* the clean input box $[0,1]^n$ and the perturbed input box $[-\varepsilon, 1+\varepsilon]^n$. Initialisation: $\mathbf{c}^{\mathrm{B}}_0 = \frac{1}{2}\mathbf{1}$, $G^{\mathrm{B}}_0 = \mathrm{diag}(\frac{1}{2}+\varepsilon)$ (`utils/perturbation_intervals.jl:1839--1843`). The intersection with Source A is applied *at every ReLU layer*, *before* the DeepZ relaxation (line 1921), so the relaxed tube narrows as Source A tightens it.

3.3 Per-copy bound integration — `intersect_per_copy_bounds`

Where. `utils/MIPVerify.jl/src/net_components/core_ops.jl:199--246`. Called by *both* the decision site (`compute_n2_relax_decision!`) and the emit site (`relu()`).

```
function intersect_per_copy_bounds(
   l_init::Real, u_init::Real,
   nn_layer::Int, nn_neuron::Int,
   m_idx::Int, k_idx::Int,
   version::AbstractString,
)::Tuple{Float64, Float64}
   l = Float64(l_init); u = Float64(u_init)
   # Source A: N1 per-copy bound + diff bound
   if !isempty(n1_neuron_bounds) && (version == "org" || version == "perturbation")
      key = (nn_layer, nn_neuron)
      if haskey(n1_neuron_bounds, key)
         (n1_u, n1_l) = n1_neuron_bounds[key]
         if !isempty(relu_diff_up_bounds) && 1 <= m_idx <= length(relu_diff_up_bounds)
              &&
            1 <= k_idx <= length(relu_diff_up_bounds[m_idx])
             u = min(u, n1_u + relu_diff_up_bounds[m_idx][k_idx])
             l = max(l, n1_l + relu_diff_down_bounds[m_idx][k_idx])
         end
      end
   end
   # Source B: shared absolute N2 hull
   if !isempty(n2_abs_up_bounds) && (version == "org" || version == "perturbation")
      if 1 <= m_idx <= length(n2_abs_up_bounds) &&
        1 <= k_idx <= length(n2_abs_up_bounds[m_idx])
         u = min(u, n2_abs_up_bounds[m_idx][k_idx])
         l = max(l, n2_abs_down_bounds[m_idx][k_idx])
      end
   end
   # ... Source C (n1 probe LP) intentionally omitted for our combo
   return (l, u)
end
```

The per-copy story: `n1_neuron_bounds` is keyed by `(neurons_names.layer, neurons_names.neuron)`, where `neurons_names.layer` runs $1..K$ during the org pass and $K{+}1..2K$ during the pert pass. The diff and Source-B arrays are indexed by `(m_idx, k_idx)` with `m_idx` resetting to $1..K$ in each pass, so they are shared between copies. This asymmetry is exactly what `advstd_techniques.tex` §4.3 calls out: only Source A varies per copy.

Emit-site call. Inside `relu()` at `core_ops.jl:295--301`:

```
if network_version == "org" || network_version == "perturbation"
   (l, u) = intersect_per_copy_bounds(
      l, u,
      neurons_names.layer, neurons_names.neuron,
      layer_counter, neurons_names.neuron,
      network_version,
   )
end
```

`layer_counter` is reset to 0 at the start of each pass (see e.g. `utils/perturbation_models.jl:142`) and incremented by 1 per ReLU layer at `core_ops.jl:739`, so `layer_counter` runs $1..K$ for each pass — matching the decision-site call argument-for-argument.

4 Tech 2 — `prev_pgd` variable hints

Where. `apply_n1_var_hints!` in `utils/help_functions.jl:598--741`; the `prev` math in `compute_varhint` at lines 507–523.

What `prev_pgd` delivers. For each surviving N_2 binary a_i (i.e. those with $l_i^{N_2} < 0 < u_i^{N_2}$ after Tech 1's elimination), compute $\texttt{hint\_val}(a_i) = (p_i \geq 0.5)\,?\,v_i^{N_1} : 1 - v_i^{N_1}$ using the prev rule, then route it through `set_start_value` filtered by PGD's three-way consensus. *No* `VarHintVal`/`VarHintPri` is written.

4.1 Step 1 — name-match N_1's primal

`help_functions.jl:619--654`. The N2 binary's name is `{nv}a_layerCount{lc}_neuronCount{nc}_{layer}_{ne` the regex `r"a_layerCount\d+_neuronCount\d+_(\d+)_(\d+)$"` extracts `(layer, neuron)` which keys directly into `layers_info_dict` (the global N_2 tightened-bounds table) and into the saved `n1_layers_info`. Confidence and other non-ReLU binaries are skipped because their names don't match the regex.

4.2 Step 2 — the prev rule for p_i

`help_functions.jl:671--699`. Two proxies:

- *Active* ($v_i^{N_1} = 1$): read $\hat{z}_i^{N_1}$ directly from N1's primal via the matching `x_rect` name — same name pattern, with the literal substring `a_layerCount` replaced by `x_rect_layerCount` once (line 681).
- *Inactive* ($v_i^{N_1} = 0$): the big-M encoding only constrains $\hat{z}_i^{N_1} \in [l_i^{N_1}, 0]$, so use the midpoint $l_i^{N_1}/2$ as a proxy (line 684).

The diff bound is looked up at line 688 via `r = (layer <= K) ? layer : (layer - K)`, mapping the 1..2K global layer back to the 1..K diff slot (the same diff applies to both copies). Then `compute_varhint` at lines 507–523 builds $\tilde{I}_i = [\hat{z}_i^{N_1} + d_i^{\downarrow},\ \hat{z}_i^{N_1} + d_i^{\uparrow}] \cap [l_i^{N_2}, u_i^{N_2}]$ and computes $p_i = \text{length}(\tilde{I}_i \cap \text{N1-side})/\text{length}(\tilde{I}_i)$, where N1-side is $(-\infty, 0]$ if $v_i^{N_1} = 0$ else $[0, +\infty)$. Degenerate (empty/point) intervals fall back to ($\texttt{hint\_val} = v_i^{N_1}$, $\texttt{pri} = 1$) at line 513.

4.3 Step 3 — PGD-consensus Start filter

help_functions.jl:702--720:

```
if mode == VH_DIRECT_PGD || mode == VH_PREV_PGD
   v_pgd_raw = JuMP.start_value(v)
   if v_pgd_raw === nothing
      JuMP.set_start_value(v, Float64(hint_val)) # fill
      n_pgd_silent_filled += 1
   else
      v_pgd_bit = Int(round(v_pgd_raw))
      if v_pgd_bit == hint_val
         n_pgd_agreed_nop += 1 # leave
      else
         JuMP.set_start_value(v, nothing) # withdraw
         n_pgd_disagreed_withdrew += 1
      end
   end
   applied += 1
else
   MOI.set(JuMP.backend(m_n2), Gurobi.VariableAttribute("VarHintVal"), JuMP.index(v),
       Float64(hint_val))
   MOI.set(JuMP.backend(m_n2), Gurobi.VariableAttribute("VarHintPri"), JuMP.index(v),
       hint_pri)
   applied += 1
end
```

The non-PGD modes (prev/direct) take the else branch and write the soft hint pair. The _pgd modes never enter that branch, so they never write VarHintVal/VarHintPri — the hint priority is discarded by design.

What survives the filter. The function tracks four counters logged at line 728: pgd-silent-filled, pgd-agreed-nop, pgd-disagreed-withdrew, and flipped (how many of the hint_vals disagreed with $v_i^{N_1}$ after the prev rule's flip). The third counter — withdrawals — represents binaries where *both* signals had an opinion but disagreed. Sweep logs typically show the silent-filled count is the largest, because PGD only sets Start on a small subset of binaries.

Survivor filter. Lines 661–664 short-circuit any binary whose tightened bound is single-signed:

```
if l_n2 >= 0 || u_n2 <= 0
   n_tier1_skipped += 1
   continue
end
```

This is defensive: relu() would not have created the binary in the first place. The counter logged at line 728 names this "Tech2-eliminated-but-survived".

Abort guard. Line 736: if applied == 0 but the model has any binaries, the function errors out before optimize! so we never silently ship a no-hint result mislabeled as prev_pgd.

5 Tech 3 — $\tau = 0.5$ BoundTightPertRelax

5.1 Decision site

`utils/n2_relax_decision.jl:23--130`. Pseudocode-faithful to `advstd_techniques.tex` §4.3. The tiered rule:

```
if max(g_org, g_pert) <= threshold # Tier 1
   relax_org, relax_pert = true, true
elseif min(g_org, g_pert) <= threshold # Tier 2
   if g_org <= g_pert
      relax_org, relax_pert = true, false # org wins on tie
   else
      relax_org, relax_pert = false, true
   end
else # Tier 3
   relax_org, relax_pert = false, false
end
```

The gap formula $g(l, u) = u \cdot |l|/(2(u - l))$ is in `_tri_gap` at `core_ops.jl:187--188` (returns 0 for single-signed intervals). The decision starts with `(l_init, u_init) = (-Inf, +Inf)` (`n2_relax_decision.jl` lines 48–49) and lets `intersect_per_copy_bounds` narrow. This is intentionally conservative: a wider starting envelope can only shrink the relaxation count.

Decision dict layout. Lines 113–123:

```
if relax_org
   n2_relax_decision[(nn_layer_org, k_idx)] = (true, false)
end
if relax_pert
   n2_relax_decision[(nn_layer_pert, k_idx)] = (false, true)
end
```

The keys distinguish copies (`nn_layer_org = m_idx`, `nn_layer_pert = m_idx + K`). The lookup site (next subsection) selects the right boolean by `network_version`.

5.2 MIP emit site

`core_ops.jl:554--605`. After `intersect_per_copy_bounds` narrowed (l, u):

```
if adv_std_n2_relax_threshold >= 0.0 &&
   (network_version == "org" || network_version == "perturbation") &&
   haskey(n2_relax_decision, (neurons_names.layer, neurons_names.neuron))
    (relax_org, relax_pert) = n2_relax_decision[(neurons_names.layer, neurons_names.
        neuron)]
    should_relax = (network_version == "org") ? relax_org : relax_pert
    if should_relax
       # emit triangle inequalities on (l, u): the SAME (l, u) the decision saw
       # ... triangle big-M-free encoding here ...
    end
end
```

The load-bearing invariant is that the emit site uses the same (l, u) that the decision evaluated. Both sites compute (l, u) via `intersect_per_copy_bounds` with arg-equivalent inputs (see §3). The triangle is therefore emitted on the exact interval whose gap was tested against τ.

5.3 Soundness

For $(m, k, \text{copy}) \notin R$: the exact big-$M$ encoding is emitted, identical to baseline. For $(m, k, \text{copy}) \in R$: the triangle inequalities are emitted on $[l_{m,k}^{N_2,\text{copy}}, u_{m,k}^{N_2,\text{copy}}]$, which covers the exact ReLU on the same interval. So $\mathcal{F}_2 \subseteq \mathcal{F}_2^{(R)}$ and $\delta_{\text{BTPR}} \geq \delta_{\text{exact}}$. (Same argument as `advstd_techniques.tex` §4.3 *Soundness.*)

6 Hyper-attack (PGD) in advstd Phase 2

Two channels. `--use_hyper_attack true` arms two distinct things in Phase 2:

1. A `Cutoff` on N_2's MIP (always installed when PGD succeeded).
2. Either MIP-Start hints (`adv_std_mip_start=true`, rejected by every safe combo) or Start values feeding the `prev_pgd/direct_pgd` consensus filter.

6.1 Subprocess invocation

`hyper_attack` at `utils/hyper_attack.jl:98--113` shells out to the Python script:

```
function hyper_attack(dataset, c_tag, c_target, token_signature,
                model_name, model_path, perturbation, perturbation_size;
                force_cpu::Bool=false)
   pre_time = @elapsed begin
      cmd_args = ["python3", "./utils/hyper_attack.py",
                "--dataset", string(dataset),
                "--source", string(c_tag-1), # 0-indexed for Python
                "--target", string(c_target-1),
                "--token", token_signature,
                "--model", model_name,
                "--model_path", model_path*"th", # .pth checkpoint
                "--perturbation", perturbation,
                "--perturbation_size", create_perturbation_string(perturbation_size)]
      if force_cpu; push!(cmd_args, "--cpu"); end
      run(Cmd(cmd_args))
      best_feasible_via_optimization = read_best_val_via_optimization(c_tag, c_target,
          token_signature)
   end
   return best_feasible_via_optimization, pre_time
end
```

The Python side (`utils/hyper_attack.py`) runs PGD for 500 iterations (`attack` at lines 120–303), keeps the top-K adversarial inputs satisfying source→target misclassification, and writes three `/tmp` files per (c_s, c_t) pair (or, on failure, a single `fail_*` file):

- `/tmp/booleans_{c-1}_{t-1}_{token}.txt` — comma-separated 0/1/-1 flags per binary (-1 = abstain).
- `/tmp/strings_{c-1}_{t-1}_{token}.txt` — matching variable names. `build_str` at lines 100–117 constructs the names: e.g. `org a_layerCount2_neuronCount0_2_5` for ReLU layer 2 / neuron 5 in the org copy, or with `perturbation` for the perturbed copy.
- `/tmp/best_val_{c-1}_{t-1}_{token}.txt` — the feasible objective value, used as the Cutoff.

In Phase 2, *both* N_1 and N_2 get their own PGD run with distinct tokens (`token_signature` for N_1 at `run.jl:860--862`, `token_signature * "_n2"` for N_2 at `run.jl:909--912`).

6.2 Hint application

`hyper_attack_hints` at `utils/hyper_attack.jl:1--35` reads the two text files, matches names against `JuMP.all_variables(m)`, and calls `set_start_value` on matched binaries (skipping `-1` abstains and unmatched names):

```
function hyper_attack_hints(m, token, c_tag, c_target)
   av = JuMP.all_variables(m)
   if isfile("/tmp/fail_"*string(c_tag-1)*"_"*string(c_target-1)*"_"*token*".txt")
      rm("/tmp/fail_"*string(c_tag-1)*"_"*string(c_target-1)*"_"*token*".txt")
   else
      # ... read booleans + strings ...
      for n in eachindex(arr_strings)
         if indexes_[n] == -1 || arr_booleans[n] == -1; continue; end
         set_start_value(av[indexes_[n]], arr_booleans[n])
      end
   end
end
```

Cutoff plumbing. Whatever PGD returned as the best feasible objective rides in `d_n2[:suboptimal_solution]`. `mip_set_attr` at `utils/mip.jl:20--34`:

```
function mip_set_attr(m, perturbation, d, timout)
   ...
   if d[:suboptimal_solution] != 0
      set_optimizer_attribute(m, "Cutoff", d[:suboptimal_solution])
   end
   ...
end
```

`Cutoff` prunes any branch whose objective is provably worse, giving Gurobi a strong dual-bound prune even when no MIP-Start is installed.

6.3 Interaction with `prev_pgd`

The `prev_pgd` consensus filter at `help_functions.jl:702--720` reads `JuMP.start_value(v)` on every surviving binary — which is exactly what `hyper_attack_hints` just wrote. The filter reconciles the two signals:

- Binaries where PGD wrote a value and the prev-rule `hint_val` agrees: PGD's value is left in place (*leave*).
- Binaries where they disagree: `set_start_value(v, nothing)` unsets PGD's value (*withdraw*).
- Binaries where PGD wrote nothing: filled with the prev-rule `hint_val` (*fill*).

So in our combo, hyper-attack contributes *(a)* the Cutoff unconditionally and *(b)* a partial Start that the consensus filter trims down to the high-confidence overlap with N_1's transferred bits. If PGD failed entirely (`fail_*` file present, every `start_value` is `nothing`), the combo degrades gracefully to "prev fills every gap" — i.e. a pure N_1-transfer Start.

Order constraint, again. `hyper_attack_hints` runs at `run.jl:999--1006`; `apply_n1_var_hints!` runs at `run.jl:1027--1034`. Reversing the order would cause PGD to overwrite the consensus filter's writes and silently break the combo's semantics.

7 VHAGaR dependencies in advstd Phase 2

Toggle. `--activate_vaghgar_deps true`. The advstd Phase-2 driver wires it into both passes: `run.jl:880--882` for the N_1 MIP and `run.jl:1007--1011` for the N_2 MIP.

Entry point. `perturbation_dependencies` in `utils/perturbation_dependencies.jl:202--232`:

```
function perturbation_dependencies(m, nn, perturbation, perturbation_size, w, h, k;
                           activation_start=1, layers_offset=nothing,
                           perturbation_var=nothing)
   layers_n = isnothing(layers_offset) ? layers_number(nn) : layers_offset
   phi_dep = fill(NaN, (1, w, h, k))
   perturbation_init_deps(phi_dep, perturbation, perturbation_size)
   activation_cnt = activation_start
   if reuse_bounds_conf.is_reuse_bounds_and_deps && activation_start == 1
      for (activation_cnt, phi_dep) in enumerate(reuse_bounds_conf.reusable_deps)
         encode_dependencies(m, layers_n, phi_dep, activation_cnt)
      end
   else
      for l in nn.layers
         if occursin("Flatten", string(typeof(l)))
            phi_dep = phi_dep |> l
         elseif occursin("Linear", string(typeof(l))) || occursin("Conv", string(
             typeof(l)))
            phi_dep_l = dep_propagation(l, phi_dep)
            phi_dep = dep_additional(m, layers_n, l, phi_dep, phi_dep_l,
                                     perturbation, perturbation_size,
                                     activation_cnt, activation_start, perturbation_var)
         elseif occursin("ReLU", string(typeof(l)))
            phi_dep = encode_dependencies(m, layers_n, phi_dep, activation_cnt)
            if activation_start == 1; push!(reuse_bounds_conf.reusable_deps, phi_dep);
                 end
            if all(isnan, phi_dep); break; end
            activation_cnt += 1
         end
      end
   end
end
```

What `phi_dep` encodes. A 4D matrix shaped like the perturbation input, holding one of $\{0, 1, -1, \text{NaN}\}$ per element:

- `0` — org and pert copies are forced equal at this neuron.
- `1` — pert dominates org (org $\leq$ pert).
- `-1` — org dominates pert.
- `NaN` — relation unknown; the encoder resolves it via a sub-LP if the upstream interval bounds permit (see below).

Per-neuron constraint emit. `encode_dependencies` at `utils/perturbation_dependencies.jl:112--200` walks each ReLU layer index `activation_cnt`. For each neuron n where both copies have a stored bound entry (`layers_info_dict[(activation_cnt, n)]` for org and `layers_info_dict[(activation_cnt + layers_n, n)]` for pert, where `layers_n = K` is the per-copy layer count):

```
if dep == 0
```

```
    @constraint(m, av[ind_o+1] == av[ind_p+1]) # x_rect equality
    if has_a_o && has_a_p
        @constraint(m, av[ind_o+2] == av[ind_p+2]) # binary equality
    end
elseif dep == 1
    @constraint(m, av[ind_o+1] <= av[ind_p+1])
    if has_a_o && has_a_p
        @constraint(m, av[ind_o+2] <= av[ind_p+2])
    end
elseif dep == -1
    @constraint(m, av[ind_o+1] >= av[ind_p+1])
    if has_a_o && has_a_p
        @constraint(m, av[ind_o+2] >= av[ind_p+2])
    end
end
```

The `ind_o + 1` and `ind_o + 2` offsets correspond to the `x_rect` (post-ReLU) and binary a that follow the pre-activation `ind_o` in the variable layout. The `has_a_o` / `has_a_p` guards (lines 124–134) check the expected variable name suffix `_{layer}_{neuron}` before emitting the binary-equality constraint — this defends against neurons whose binary was *eliminated* by Tech 1 (`l` $\geq$ `0` or `u` $\leq$ `0`) or *relaxed* by Tech 3 (triangle, no binary). When the binary is gone, only the `x_rect` constraint is emitted, which remains sound because the relaxed encoding still produces a valid z^+.

Sub-LP for unknown relations. When `dep == NaN` and the upstream bounds don't immediately resolve it (lines 152–159 handle the easy $l_{\text{org}} \geq u_{\text{pert}}$ and $l_{\text{pert}} \geq u_{\text{org}}$ cases), `encode_dependencies` introduces an auxiliary objective on $\text{av}[\text{ind}_o + 1] - \text{av}[\text{ind}_p + 1]$ and calls `optimize!(m)` to compute a sound lower or upper bound on the diff (lines 161–192). The Cutoff is set to 0 to terminate the sub-LP early once a definitive sign is found. If the resulting sub-bound is sufficiently far from zero (`l_diff > -non_equality_tolerance` or `u_diff < non_equality_tolerance`, where the default tolerance is `1e-4`), the dependency is ratified and the corresponding constraint pair is emitted. This sub-LP machinery is unchanged from standard mode and is independent of the advstd techniques.

Interaction with the combo. `phi_dep` is built from the *same* `layers_info_dict` that `relu()` populated during `get_model`. Because Tech 1 already injected N_1's tighter bounds and Tech 3 may have eliminated or relaxed some neurons, the dep encoder sees the *post-Tech* view of (u, l). This works correctly because:

- Tighter bounds make the easy resolution at lines 152–159 fire more often, reducing sub-LP work.
- Eliminated binaries trigger the `has_a_o`/`has_a_p` guard at lines 133–134, suppressing the binary-equality constraint that would otherwise reference a non-existent `av[ind+2]`.
- Relaxed binaries (Tech 3 triangle) similarly miss `av[ind+2]` and the guard suppresses the binary constraint.

Caching. First call (`activation_start == 1`) pushes each layer's resolved `phi_dep` into `reuse_bounds_conf.r`. On subsequent class-pair iterations within the same Phase 2 the encoder reuses these resolved depending-relations and skips the sub-LP step (lines 209–212). The cache lives in `help_functions.jl`'s globals; it is cleared between Phase-2 runs.

8 Reproducing the safe combo end-to-end

8.1 Phase 1 — solve N_1 once

```
julia run.jl --mode advanced_standard_n1 \
  --dataset mnist --model_name 3x10 \
  --model_path  /path/to/N1/model.p \
  --model_path2 /path/to/N2/model.p \
  --perturbation translation --perturbation_size "1,1" \
  --ctag 1 --ct "4,5,6" --timout 1800 \
  --n1_state_dir /path/to/n1_state \
  --use_hyper_attack       true \
  --activate_vaghgar_deps  true \
  --use_perturbed_intervals true
```

This solves N_1's standard MIP (with PGD warmstart, VHAGaR-deps, and perturbed intervals all active) and serialises `n1_layers_info`, the primal solution, and the diff-bound inputs into `n1_state`. Repeat per (c_s, c_t) pair.

8.2 Phase 2 — the safe combo

```
julia run.jl --mode advanced_standard_n2 \
  --dataset mnist --model_name 3x10 \
  --model_path  /path/to/N1/model.p \
  --model_path2 /path/to/N2/model.p \
  --perturbation translation --perturbation_size "1,1" \
  --ctag 1 --ct "4,5,6" --timout 1800 \
  --n1_state_dir /path/to/n1_state \
  --use_hyper_attack             true   \   # Cutoff + PGD Start (consumed by
     prev_pgd)
  --activate_vaghgar_deps        true   \   # Section 6
  --use_perturbed_intervals      true   \   # required for sound
     BoundTightPertRelax
  --adv_std_bound_tightening     true   \   # Tech 1, prerequisite for tau and
      prev_pgd
  --adv_std_zono_bounds          true   \   # zono Source A + B
  --adv_std_n2_relax_threshold   0.5    \   # Tech 3, tau = 0.5
  --adv_std_var_hint             prev_pgd   # Tech 2, prev rule + Start
     consensus
```

Healthy-stdout sanity checks.

- `compute_n2_relax_decision!: tau=0.5, both=... org-only=... pert-only=... none=...` `(plus ... stable neurons)` — non-zero `both + org-only + pert-only`.
- `apply_n1_var_hints!: mode=PREV_PGD, Start-consensus on N of M N2 binaries (pgd-silent-f` `pgd-agreed-nop=..., pgd-disagreed-withdrew=..., flipped=...)` — no abort, `N == M`.
- Filename suffix on the saved CSV: `..._N2_advStd_BoundTightPertRelax0.5_zonoBounds_varHintPrevPGD`
- Final `delta_n2` is finite and the run terminates `OPTIMAL` or `TIME_LIMIT` with a finite incumbent.

Unsoundness guard. The driver aborts before encoding if `--adv_std_n2_relax_threshold` > 0 is paired with `--use_perturbed_intervals false`, because the soundness argument for tiered BoundTightPertRelax requires the inter-copy coupling to remain intact (`run.jl:673--680`).